

P³Forecast: Personalized and Adaptive Cloud Workload Prediction via GAN-based Federated Data Augmentation

Xin Yong, *Member, IEEE*, Li Yan, *Member, IEEE*, Yu Kuang, *Member, IEEE*, Zhuozhao Li, *Member, IEEE*, Chao Shen, *Fellow, IEEE* and Xingwei Wang

Abstract—To comply with privacy regulations and self-protection from cloud outages, increasingly more users are leveraging multiple cloud providers for service deployment. However, due to the isolation of highly Non-Identically and Independently Distributed (Non-IID) workload datasets and deficiencies of existing methods as reflected in our experimental studies, no cloud providers can single-handedly capture the workload patterns of such users for accurate workload prediction. Accordingly, we propose *P³Forecast*, a Personalized Privacy-Preserving Cloud workload prediction framework based on Federated Generative Adversarial Networks (GANs), which allows cloud providers with Non-IID workload data to collaboratively train workload prediction models as preferred while protecting privacy. We first design a data synthesis quality assessment method based on Dynamic Time Warping (called *pattern-aware DTW*), which is insusceptible to time series length and reliable for the comparison of temporal patterns. By using *pattern-aware DTW* as the model aggregation weights, we adopt the Federated Learning (FL) of a GAN model for the augmentation of IID workload training datasets per cloud provider. Then, we further design a post-training method of local workload prediction models, which consists of a query mechanism based on comprehensive evaluation of data synthesis informativeness and an adaptive learning rate adjustment strategy for stable convergence. Extensive experiments driven by real-world workloads demonstrate that compared with the state-of-the-art, *P³Forecast* improves workload prediction accuracy by 23.7%-64.5% on average over all cloud providers, while ensuring the fastest convergence in Federated GAN training.

Index Terms—Cloud Workload Prediction, Federated Learning, Generative Adversarial Networks

1 INTRODUCTION

With the booming of cloud computing, there is an increasing demand for predictive orchestration of cloud resources. However, due to the popularity of new cloud applications, workloads are becoming increasingly heterogeneous and dynamic, which hinders the accurate prediction of cloud workloads [1]–[3].

Multiple traditional approaches have been developed for workload prediction of individual cloud providers [4]–[10]. Generally, they utilize various Machine Learning (ML) techniques, such as Autoregressive Integrated Moving Average (ARIMA), Deep Neural Networks (DNN), to capture the complex patterns and high-dimensional features of workloads. However, the growing complexity and dynamics of workloads from emerging applications (e.g., large language

model serving systems) present increasingly multifaceted challenges [11], [12]. Constrained by traditional prediction paradigms and isolated workload databases, cloud providers are finding that their localized workload data and empirical insights lag significantly behind the rapid evolution of actual services [13], resulting in a variety of prediction bottlenecks. For example, public cloud providers such as *Alibaba* have extensive experience in handling ML workloads, but lack sufficient data to learn patterns associated with scientific computing workloads [3]. Under such cases, traditional approaches are unable to versatily predict workloads covering various service categories, which makes it challenging for users to leverage the workload prediction results for optimal deployment of their applications across multiple cloud platforms [14], [15].

Nevertheless, it is neither practical nor secure to share workload traces across providers due to the high expense of network transmission and risk of data privacy leakage [16], [17]. Federated Learning (FL) [18], which enables collaborative ML model training without exposing participants' privacy-sensitive data, shines a light on establishing a versatile workload prediction framework via collaboration across providers [14], [15], [19]. Despite this, our preliminary studies (Section 3.4) show that the workloads of different cloud providers are often highly Non-Identically and Independently Distributed (Non-IID) due to their different cloud computing environments and diverse business involvements, and may cause significant performance drop in FL training [20], [21].

Manuscript received 15 October 2025; revised 10 May 2026; accepted 10 June 2026. This work was supported by the National Key R&D Program of China under Grant No. 2022YFB4500800, the National Natural Science Foundation of China under Grant No. 62103325, and the Shaanxi High-Level Talent Program under Grant No. 2021QCYRC4-26. This article was presented in part at the IEEE IPDPS 2025 [DOI: 10.1109/IPDPS64566.2025.00038] (Corresponding author: Li Yan.)

Xin Yong (xin_yong@stu.xjtu.edu.cn), Li Yan (li.yan.88@xjtu.edu.cn), Yu Kuang (yooki.k@stu.xjtu.edu.cn) and Chao Shen (chaoshen@mail.xjtu.edu.cn) are with the School of Cyber Science and Engineering, Xi'an Jiaotong University, Xi'an, Shaanxi 710049 China.

Zhuozhao Li (lizz@sustech.edu.cn) is with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China.

Xingwei Wang (wangxw@mail.neu.edu.cn) is with the School of Computer Science and Engineering, Northeastern University, Shenyang 110819, China.

In recent years, numerous approaches have been proposed to tackle the issues caused by Non-IID data in FL systems [21]–[28]. Among them, the methods leveraging shared Generative Adversarial Networks (GANs) to augment FL training data, known as Virtual Homogeneity Learning (VHL), [23], [24], [29] stand out for their faster convergence and strong generalization performance on both training and testing datasets. However, these methods mostly focus on tasks involving spatial data (e.g., image classification task), and are therefore unsuitable for cloud workload prediction, which relies on time series data with abundant temporal features. Moreover, to augment the FL training data, these methods require participants to share the base noise that covers all heterogeneous workload features (e.g., CPU usage, requested memory amount) across cloud providers, which is against the privacy regulations of most cloud providers [14], [15].

A possible solution to the problems mentioned above is to train the GAN model in a FL manner [30]–[35]. Thus, each client can use the trained GAN model to synthesize its own IID training data. However, different from the model aggregation strategy in classic FL tasks, of which model updates are generally averaged or weight-averaged by data quantity or data distribution discrepancy, the contribution of clients in the FL training of time series GAN models should be scored by the quality of the synthesized time series data for faster convergence and better data synthesis capability [33], [34]. Although Dynamic Time Warping (DTW) has been widely utilized to assess temporal data synthesis quality in GAN-based data augmentation [36], [37], our experimental studies in Sections 3.5 and 3.6 demonstrate that DTW is unreliable for workload data synthesis quality assessment, and existing methods are inefficient due to the lack of personalization in data augmentation.

Considering that temporal features are vital for effective workload data augmentation under Non-IID settings, while existing methods suffer from unreliability in data synthesis quality assessment and inefficiency in data augmentation, we propose *P³Forecast*, a **P**ersonalized **P**rivacy-**P**reserving Cloud Workload Prediction framework based on Federated GAN, which allows cloud providers with heterogeneous workload data to collaboratively train workload prediction models according to their preference while protecting privacy. This method extends our earlier work [38] with key improvements in the post-training of workload prediction models. Specifically, we first incorporate the measurement of the first-order difference between time series and normalization into DTW to ensure that our proposed data synthesis quality assessment method (called the *pattern-aware DTW*) is insusceptible to time series length and reliable for the comparison of temporal patterns. Then, by using the *pattern-aware DTW* as the weight of model updates from the cloud providers, we adopt FL for the collaborative training of a GAN model, which can be utilized by each cloud provider to synthesize its own IID training data. Finally, we design a post-training method of local workload prediction models, which consists of a query mechanism based on comprehensive evaluation of data synthesis informativeness and an adaptive learning rate adjustment strategy for stable convergence. Unlike most existing FL personalization methods that require consistent model architectures across clients,

P³Forecast allows each cloud provider to specify its workload prediction model architecture as preferred, which is especially suitable for cloud providers with heterogeneous model preferences and data augmentation preferences. The contributions are summarized as follows:

(1) We conduct extensive experimental studies on large-scale workload datasets collected from seven cloud clusters or centers to demonstrate the Non-IID nature of cloud workloads and deficiencies of existing methods in realizing the collaborative training of workload prediction models.

(2) We propose *P³Forecast*, a personalized privacy-preserving cloud workload prediction framework based on Federated GAN. The framework utilizes a novel DTW-based data synthesis quality assessment method for the aggregation of Federated GAN models, which is both insusceptible to sequence length and reliable for temporal pattern comparison.

(3) We further design a personalized post-training method for workload prediction models in *P³Forecast*, which employs a synthesis data query mechanism based on comprehensive evaluation of data informativeness and an adaptive learning rate adjustment strategy for personalized data augmentation and stable model training.

(4) We conduct detailed experiments on real-world cloud workloads. Compared with the state-of-the-art, *P³Forecast* drastically improves workload prediction accuracy by 23.7%–64.5% on average over all cloud providers, while ensuring the fastest convergence in Federated GAN training.

The rest of this paper is organized as follows. Section 2 summarizes related works. Section 3 presents preliminaries and motivations. Section 4 introduces the detailed design of *P³Forecast*. Section 5 presents performance evaluation results. Section 6 concludes the paper with future remarks.

2 RELATED WORK

Traditional Cloud Workload Prediction. The previously proposed cloud workload prediction methods can be roughly classified into two categories: regression-based [4], [5] and learning-based [6]–[8]. Calheiros *et al.* [4] proposed an ARIMA-based approach to predict future workloads generated by web applications requests. Kim *et al.* [5] proposed to ensemble multiple predictors for cloud workload prediction. Moreover, several studies have applied deep learning models, including Long Short-Term Memory (LSTM) [6] and Gated Recurrent Unit (GRU) [7], [8] to cloud workload prediction. In addition, recent studies have increasingly explored hybrid architectures for time-series forecasting, where multi-scale patch blocks and mixing mechanisms are integrated to better capture complex temporal patterns. In addition, recent time-series forecasting studies have increasingly adopted hybrid architectures that integrate multi-scale [9] or multi-resolution [10] representations. However, due to limited and isolated data of individual cloud providers, these methods cannot establish a versatile workload prediction model capable of accurately predicting workloads covering various service categories.

Data Heterogeneity in FL. To mitigate client drift caused by data heterogeneity, regularization-based methods, such as SCAFFOLD [21], FedProx [22] and pFedMe [27] have been proposed to align the optimization of local models with the

global objective. From the perspective of data augmentation, some methods utilize shared GANs to synthesize data for knowledge distillation [23] or alleviating dataset shift [24]. Additionally, to mitigate the impact of Non-IID data, Zhou *et al.* [25] proposed a framework that clusters clients by their computation and communication capabilities, and utilizes deep Reinforcement Learning to intelligently select a subset of clients from each cluster. Unlike clustering-based methods, PFedCS [28] adaptively groups clients by measuring distances between local classifier parameters, thereby generating customized classifiers for fine-grained collaboration among clients with similar data distributions. However, these methods mostly focus on spatial data tasks, and thus cannot be applied to cloud workload prediction, of which temporal features are crucial. Moreover, the sharing of workload features across cloud providers is against the privacy regulations of most cloud providers [14], [15].

Federated GAN for Data Augmentation. The FL training of GAN models has been proposed for IID data augmentation. Hardy *et al.* [30] proposed a distributed GAN training method on clients with IID data. Rasouli *et al.* [31] proposed to use two time-scale learning rates for the generator and discriminator of GAN. To address Non-IID issues in distributed GAN training, Guerraoui *et al.* [32] proposed to utilize the Kullback-Leibler distance between local and global label categories as the FL aggregation weight. Zhang *et al.* [33] proposed a FL framework for the collaborative training of GANs among cloud providers, edge servers, and devices with Non-IID data. Li *et al.* [34] proposed an improved FL-GAN framework that weighs clients' local GAN model updates by their spatial data synthesis quality. Ma *et al.* [35] proposed a GAN-based unbiased FL scheme to mitigate Non-IID issues. However, these Federated GAN training methods are designed for spatial data tasks, thus cannot be applied for the FL training of time series GAN models due to the lack of a reliable metric for time series data synthesis quality assessment.

3 PRELIMINARIES AND MOTIVATIONS

In this section, we present the detailed observations that motivate our system design. The datasets are collected from the following seven cloud clusters or centers:

- **Alibaba dataset** [39]. It is collected from a large cluster with over 4,000 machines in Alibaba data centers, spanning a period of 8 days in 2018, with a sampling period of ten seconds. It contains the logs of Alibaba's production workloads, which record workload information such as `cpu_util_percent`, `mem_util_percent`, etc.
- **Microsoft dataset** [40]. It is collected from Microsoft's Philly clusters with over hundreds of machines, spanning from August to December of 2017, with a sampling period of one minute. It contains the logs of >96,000 deep learning jobs, recording workload information such as `cpu_util`, `mem_util`, etc.
- **Google dataset** [41]. It is collected from Google production clusters with over 52,000 machines, which covers a 1-month time frame in 2011, with a sampling period of five minutes. It contains the logs of Google's production workloads, which record workload information such as `job_ID`, `mean_CPU_usage_rate`, etc.

- **Alibaba-AI dataset** [42]. It is collected from a large production cluster with over 6,500 GPUs (on ~1,800 machines) in Alibaba Platform for Artificial Intelligence, spanning from July to August of 2020, with a sampling period of ten minutes. It records workload information such as `cpu_utilization`, `memory_utilization`, etc.

- **HPC datasets.** They are collected from three high performance computing cloud service centers (*HPC-KS*, *HPC-HF* and *HPC-WZ*) for academic research, where the *HPC-KS* dataset spans from January to February of 2022 with a sampling period of ten minutes, the *HPC-HF* dataset spans September in 2022 with a sampling period of five minutes, and the *HPC-WZ* dataset spans from January to October of 2022 with a sampling period of half an hour. They contain the logs of scientific computing, large-scale optimization or ML algorithms, which record workload information such as anonymized `job_cpu_time`, `job_mem_used`, etc.

3.1 Dynamic Time Warping

Given time series $\mathbf{x} = \{x_1, \dots, x_i, \dots, x_n\}$ and $\mathbf{y} = \{y_1, \dots, y_j, \dots, y_m\}$, the DTW algorithm first calculates the distance matrix $\mathbf{D} = \{d(i, j) | 1 \leq i \leq n, 1 \leq j \leq m\}$, where $d(i, j)$ is the Euclidean distance between x_i and y_j . DTW aims to find the optimal path from $(1, 1)$ to (n, m) with the minimum cost, which can be represented as the following recurrence and solved by dynamic programming:

$$\tilde{d}(i, j) \leftarrow d(i, j) + \min\{\tilde{d}(i-1, j), \tilde{d}(i, j-1), \tilde{d}(i-1, j-1)\} \quad (1)$$

where $\tilde{d}(i, j)$ represents the cumulative distance of the optimal path from $(1, 1)$ to (i, j) . Thus, the cumulative distance is the sum of the distance between current elements (i.e., $d(i, j)$) and the minimum of the cumulative distances of the neighboring points (i.e., $\tilde{d}(i-1, j)$, $\tilde{d}(i, j-1)$ and $\tilde{d}(i-1, j-1)$). Similarly, we can finally obtain the cumulative distance matrix $\tilde{\mathbf{D}} = \{\tilde{d}(i, j) | 1 \leq i \leq n, 1 \leq j \leq m\}$, where $\tilde{d}(n, m)$ is the DTW distance between $\mathbf{x}$ and $\mathbf{y}$. Although DTW reflects the heterogeneity between workload sequences, it has significant deficiencies when assessing data synthesis quality, as elaborated in Section 3.5.

3.2 Federated GAN

Federated GAN integrates FL with GAN to train a GAN model across distributed clients with Non-IID local data and privacy constraints [31]. Specifically, suppose that each of the K clients holds a regular GAN as the local model. Federated GAN training aims to build a mapping function that maps a random noise sub-space $\mathbf{z}$ from randomized space $\mathbf{Z}$ into the overall workload data space of the clients $\mathbf{X} = \cup_{k=1}^K \mathbf{x}_k$, where $\mathbf{x}_k$ is the workload sub-space of the k -th client. Thus, the objective function of the minimax game between the discriminator D and the generator G is:

$$\begin{aligned} \min_G \max_D V(G, D) &= \sum_{k=1}^K \alpha_k (\mathbb{E}_{x \sim p(\mathbf{x}_k)} [\log D_k(x)] + \\ &\quad \mathbb{E}_{z \sim p(\mathbf{z})} [\log(1 - D_k(G_k(z)))] \quad (2) \\ \text{s.t. } \sum_{k=1}^K \alpha_k &= 1 \end{aligned}$$

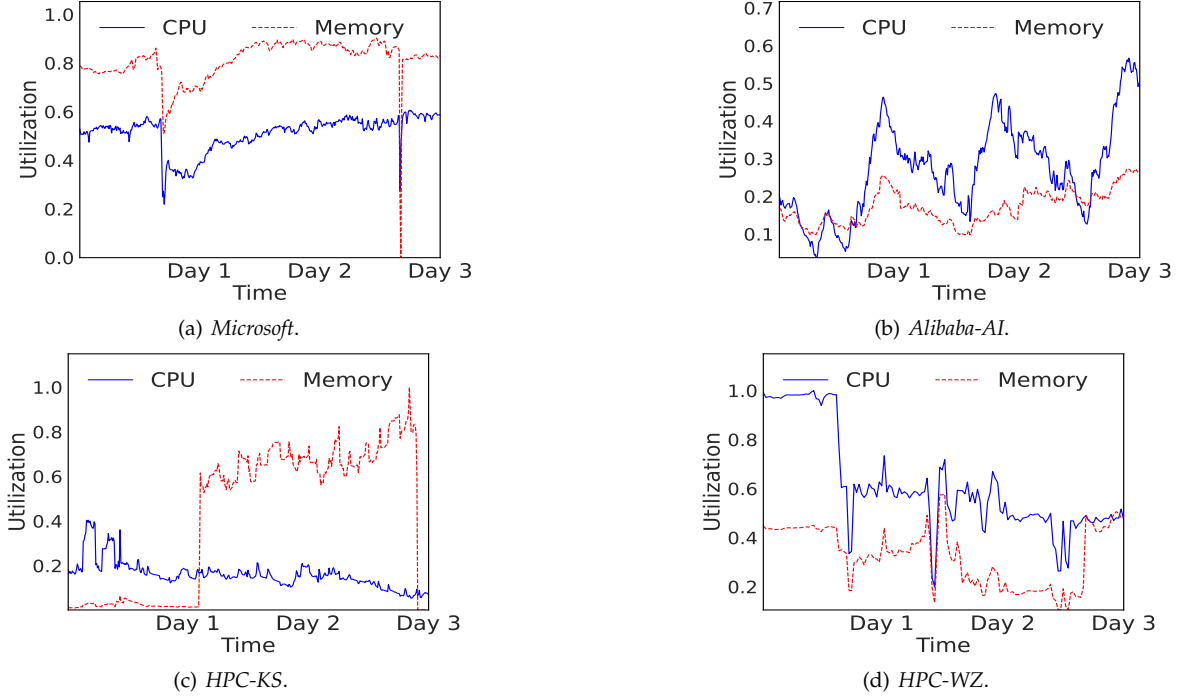


Fig. 1: Workloads of CPU and memory in four cloud providers.

where α_k represents the model update weight of the k -th client, and D_k and G_k are the discriminator and generator of the k -th client, respectively. $p(\mathbf{x}_k)$ is the local data distribution of the k -th client, while $p(\mathbf{z})$ is the noise distribution. Albeit effective in synthesizing IID data, existing Federated GAN training methods, lacking consideration of temporal features, are unsuitable for time series GAN model training.

3.3 Compatibility with Privacy Protection Mechanisms

P^3 Forecast follows the standard FL update and aggregation workflow, allowing for seamless integration with existing privacy protection techniques. For example, by applying our prior work on Batch/Selective Homomorphic Encryption (e.g., AdaptiveBatchHE [43], SHE-LoRA [44]) or Adaptive Local Differential Privacy (AdapLDP [45]), cloud providers can achieve encrypted or obfuscated model aggregation with marginal computation overhead and minimal performance loss, while preventing the server from directly accessing sensitive local information. Therefore, when stronger privacy protection is required in practice, these existing mechanisms can be incorporated into P^3 Forecast with negligible migration overhead. The detailed integration and evaluation of these techniques are beyond the scope of this paper and are left for future work.

3.4 Heterogeneous Temporal Features of Workload Data

Since the seven cloud providers have different cloud resource management architectures, the features they utilize for resource representation differ significantly. For example, *Microsoft* uses `cpu_util` to directly represent the CPU utilization of machines during sampling, while *Google* only records the CPU utilization per job. Hence, we need to combine *Google*'s jobs by timestamp, and deduce the overall CPU utilization of all its machines.

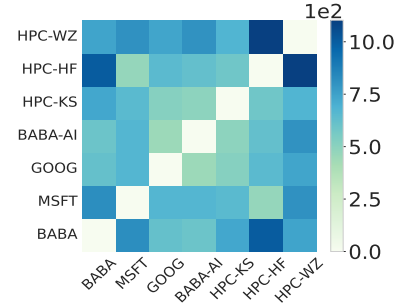


Fig. 2: Heat map matrix of DTW between CPU utilization time series.

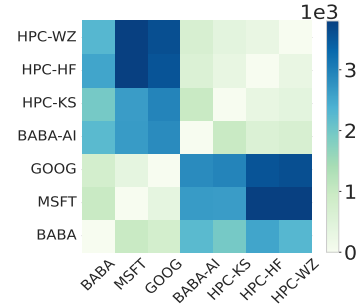


Fig. 3: Heat map matrix of DTW between memory usage time series.

In addition, even for the same resource category, different cloud providers may use different feature names. For example, *Alibaba* only uses `mem_util_percent` to represent memory utilization, while *Google* uses three features: `canonical_memory_usage`, `assigned_memory_usage` and `maximum_memory_usage`. Therefore, to illustrate the heterogeneous temporal features of workloads, we first preprocess the datasets via aligning the representations of resources, removing redundant features and min-max normalization.

Finally, we obtain several workload time series of the seven cloud providers for further temporal feature heterogeneity analysis. Figure 1 illustrates the CPU utilization

and memory utilization time series of four cloud providers over three days. It can be seen that within randomly selected observation windows, the workloads of various cloud providers demonstrate significant heterogeneity regarding their peak-and-valley patterns and cyclical trends. To more accurately confirm the data heterogeneity across the time series of all the seven cloud providers, we further calculate the DTW distances between the CPU utilization and memory utilization time series of every pair of them, which are illustrated as heat map matrices in Figure 2 and Figure 3, respectively. The darker color that the square corresponding to a pair of two cloud providers has, the more dissimilar the two cloud providers' workload time series are. We can see that the workload time series between most pairs of cloud providers are highly dissimilar (i.e., Non-IID) in terms of DTW, especially for the CPU utilization.

Challenge 1: given that the workload time series of different cloud providers are Non-IID in temporal features, while most existing FL solutions focus on spatial data, how to design a federated data augmentation method capable of synthesizing IID time series data?

3.5 Deficiency of DTW in Assessing Data Synthesis Quality

As indicated in previous methods for federated IID data augmentation via GAN, the model updates should be weighed by data synthesis quality [33], [34]. Although DTW can reflect the heterogeneity between workload time series, it has two significant deficiencies when applied for assessing data synthesis quality: 1) DTW is susceptible to the length of time series, and 2) DTW overlooks the comparison of patterns between time series, which may make it unreliable for measuring the similarity of temporal patterns.

We first use a toy example to illustrate the intrinsic reason of the first deficiency. From the description of datasets, we can see that the workload time series of different cloud providers generally have various lengths due to different sampling frequencies and total time durations. Following these observations, we prepare two sets of signals that have different lengths due to the same factors. Specifically, the first set of signals are generated by functions $y_1 = \sin 2\pi t$ and $y_2 = 0.1 \sin 2\pi t$, with a sampling frequency of 50 and an end point at $t = 2$, as shown in Figure 4(a). The second set of signals are generated by functions $y'_1 = \sin 2\pi t$ and $y'_2 = 0.8 \sin 2\pi t$, with a sampling frequency of 500 and an end point at $t = 2.5$, as shown in Figure 4(b). Intuitively, the similarity between y'_1 and y'_2 is higher than that between y_1 and y_2 as the amplitude difference between y'_1 and y'_2 is obviously much lower than that between y_1 and y_2 . However, the calculated DTW distance between y_1 and y_2 is 57.75, while the DTW distance between y'_1 and y'_2 is 72.58, which contradicts the straightforward intuition. This is primarily because that DTW measures the sum of absolute differences between the time series, so the measurement result is dependent on the length of the time series. Suppose the lengths of two sequences for DTW calculation are n and m . Possible solutions for fairly assessing data synthesis quality across cloud providers include normalizing DTW by $n + m$, $\max\{n, m\}$ or the warping path. In our case, since

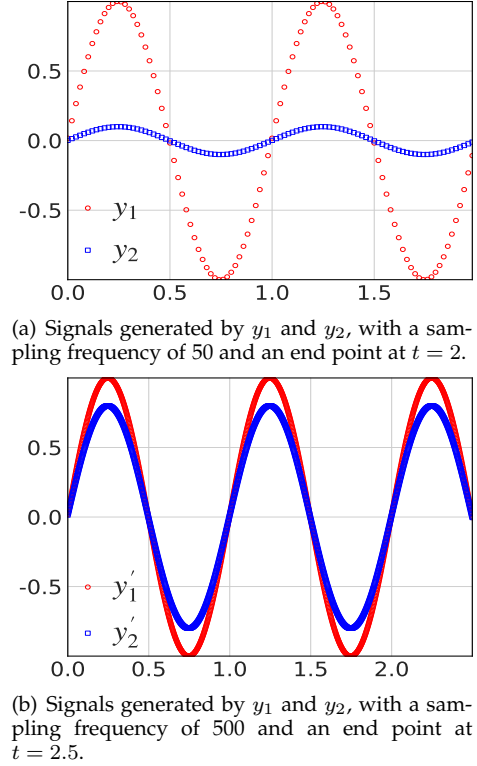


Fig. 4: Susceptibility of DTW to the length of time series.

we set that the synthesized and original workload sequences have equal lengths (i.e., $n = m$), we simply normalize their DTW by $\max\{n, m\}$ (i.e., unit-length DTW distance). For example, the unit-length DTW distance between y'_1 and y'_2 is 0.058, while the unit-length DTW distance between y_1 and y_2 is 0.578, which matches the intuitive observation.

For the second deficiency, we use two additional toy examples to illustrate the intrinsic reason. In the following, the sampling frequency and the end point of all signals are 50 and $t = 1$, respectively. Suppose we have a set of signals generated by functions $y_1 = 8(t - 0.5)^2 - 1$, $y_2 = 0.5 \cos 2\pi t$ and $y_3 = 0.5 \cos 2\pi t + 0.6$, as shown in Figure 5(a). Intuitively, the similarity of temporal patterns between y_1 and y_2 is approximate as that between y_1 and y_3 , as y_3 is actually y_2 displaced by 0.6 along the Y-axis. However, the DTW distance between y_1 and y_2 is 11.14, while the DTW distance between y_1 and y_3 is 29.94. This is primarily because that as a distance metric, the DTW algorithm essentially computes the distance from the matching of similar elements between time series, but overlooks the comparison of patterns. Traditional methods [46]–[49] generally utilize the Z-score normalization to eliminate the influence of displacement: $\bar{y} = \frac{y - \mu}{\delta}$, where y and $\bar{y}$ are the original and normalized time series, respectively, μ and δ are the mean and standard deviation of y , respectively. For example, as illustrated in Figure 5(b), the Z-normalized y_2 and y_3 (denoted as $\bar{y}_2$ and $\bar{y}_3$) completely overlap, of which DTW distances to y_1 are both 10.78.

However, Z-score normalization does not work in all cases. Specifically, suppose in Figure 5(c), we have another set of signals for pattern similarity comparison with y_1 , which are generated by functions: $y_4 = 0.5 \cos 2\pi t$ and $y_5 = \cos 2\pi t$. As we can see in Figure 5(d), although the patterns of y_4 and y_5 are obviously quite different, the Z-

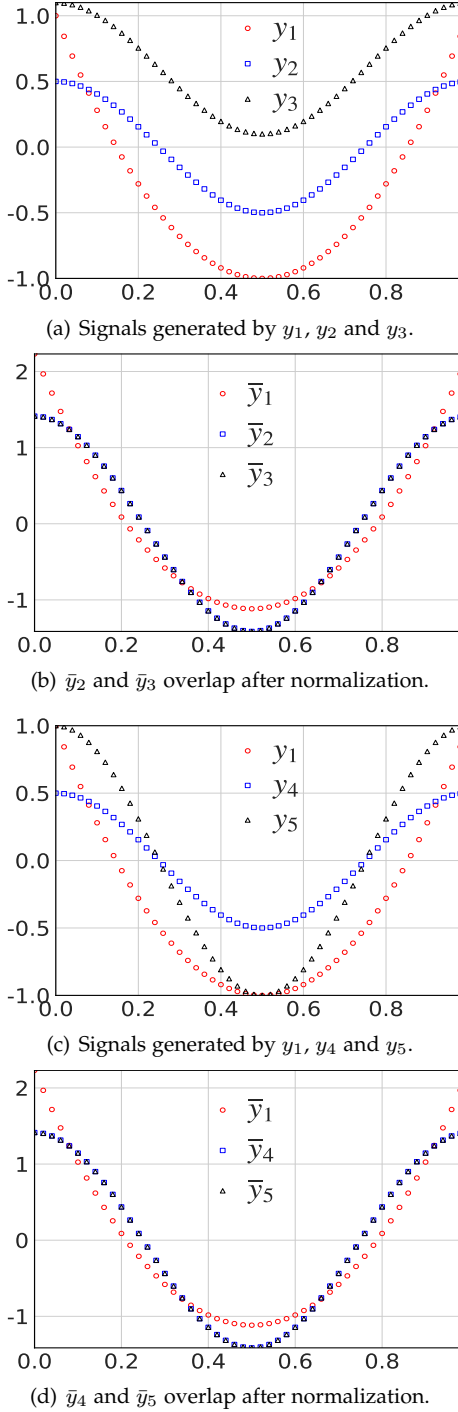


Fig. 5: Unreliability of DTW in measuring temporal pattern similarity. normalized y_4 and y_5 (denoted as $\bar{y}_4$ and $\bar{y}_5$) also completely overlap, which is incorrect. This is mainly because Z-score normalization scales the original time series amplitude by mean and standard deviation, which can inadvertently collapse distinct patterns into the same waveform.

Challenge 2: given the deficiencies of DTW, how to fairly assess data synthesis quality in a manner that is robust to sequence length discrepancy across cloud providers and reliable for temporal pattern comparison?

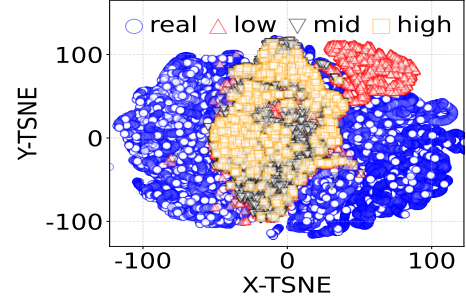


Fig. 6: Dimension reduction results of t-SNE.

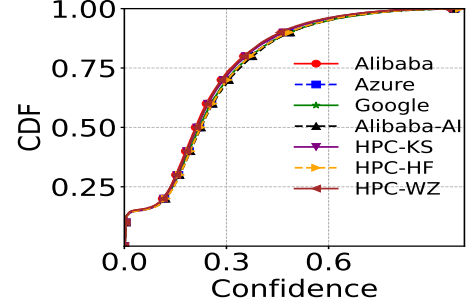


Fig. 7: CDF of discriminators' confidence scores.

3.6 Impact of Training Dataset Distribution and Size on Workload Prediction Model Convergence

Generative Adversarial Active Learning (GAAL) is effective in augmenting data according to specific criteria of the learner [50]–[53], which is potentially suitable for the personalization of workload prediction model per cloud provider. Specifically, it uses GANs to synthesize informative data samples that are adapted to the workload prediction model. Then, the synthesized data is queried by a certain criterion and added back to the training dataset to update the workload prediction model. This protocol is executed iteratively until the pre-defined termination conditions are reached.

Obviously, the query effectiveness of GAAL depends on whether the most informative samples can be queried for improving model performance. While federated GAN enables each cloud provider to train a generator that incorporates global knowledge, discrepancies in data distributions across providers and noise in personalized weights (i.e., α_k) may introduce distributional bias during data synthesis, thereby compromising the informativeness of the generated samples. Considering that the discriminator network is critical to the quality of data synthesis in GAN, we take an in-depth analysis on the effectiveness of its confidence score in assessing the informativeness of synthesized data samples, which may help the design of the query mechanism in GAAL and mitigate the above issue.

Specifically, we first use the CPU utilization and memory utilization workload data of all cloud providers to train a TimeGAN model [54] for workload data synthesis. Then, we collect the confidence scores output by the discriminator, which are classified into three categories: low confidence [0, 0.3], medium confidence (0.3, 0.6], and high confidence (0.6, 1]. Subsequently, we apply the t-SNE (t-distributed Stochastic Neighbor Embedding) [55] dimension reduction method on each category of synthesized data and real data across all cloud providers to analyze their difference, which

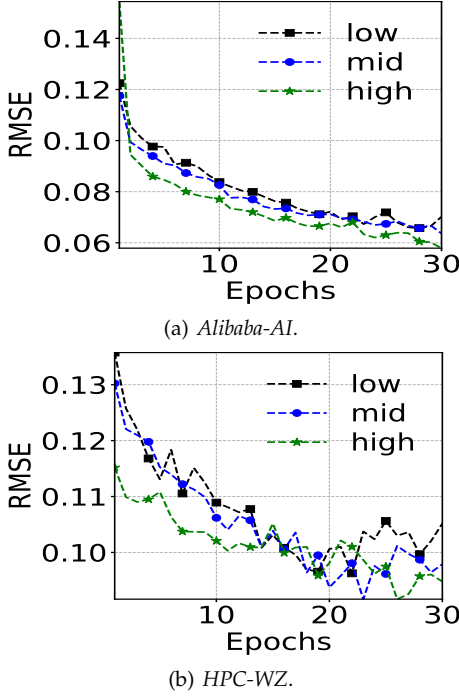


Fig. 8: Convergence under varying confidence categories.

is illustrated in Figure 6. We can see that the distribution of the synthesized data with higher confidence scores is much more similar to that of the real data, whereas the distribution of those with lower confidence scores significantly deviates from that of the real data. Meanwhile, Figure 7 shows the Cumulative Distribution Function (CDF) of the discriminators' confidence scores on all the cloud providers. We can see that for all the cloud providers, only $< 20\%$ of the synthesized data samples have confidence scores above 0.5. This means that not all data samples synthesized by the federated GAN model are sufficiently realistic from the perspective of discriminators.

Furthermore, we analyze the impact of the confidence of synthesized data on model convergence via a GAAL experiment. Specifically, we add the synthesized data samples to the training dataset of each cloud provider as in GAAL. The ratio between the amount of synthesized data samples $|G_k(z)|$ and the amount of real data samples $|\mathbf{x}_k|$ on the k -th cloud provider is denoted as $\varphi_k = \frac{|G_k(z)|}{|\mathbf{x}_k|}$. We train a GRU network with $\varphi_k = 1$ for each category of synthesized data samples on *Alibaba-AI* and *HPC-WZ*, respectively, and calculate the Root Mean Square Error (RMSE = $\sqrt{\frac{1}{n} \sum_{i=1}^n (\text{Predicted}_i - \text{Actual}_i)^2}$) of the prediction results. The GRU network consists of one input layer with the size of 2, one hidden layer with 512 hidden units, and one output layer with the size of 2. The number of training epochs is 30, and the learning rate is 0.001. The testing dataset is built by extracting a fixed portion of local workload data samples from each cloud provider. Figure 8 shows the change of RMSE along with the training progress in the two cloud providers under varying confidence categories. We find that synthesized data with higher confidence scores leads to faster model convergence on both cloud providers, implying that confidence score is effective for identifying informative synthesized data samples.

Based on the above observations, it is reasonable to

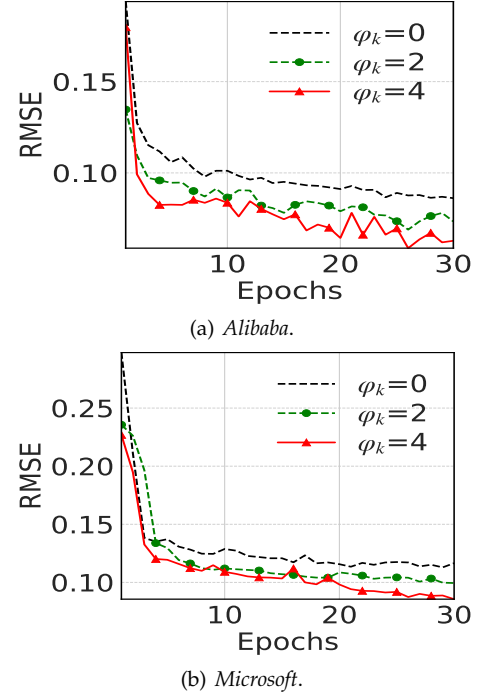
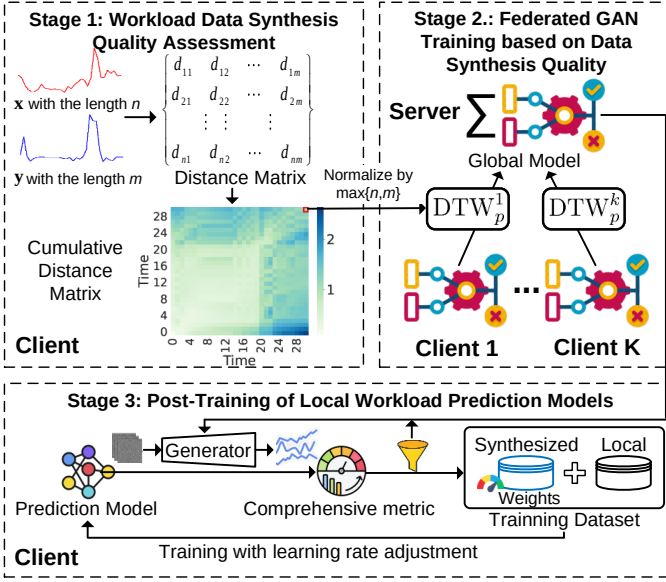


Fig. 9: Convergence under varying amounts of synthesized data.

infer that the amount of synthesized data samples may also be pivotal to the training and performance of workload prediction models. To verify our conjecture, we also conduct a GAAL experiment, but with varying ratios of synthesized data samples. Specifically, we train a GRU network under $\varphi_k = 0, 2, 4$, and calculate the RMSE values of the prediction results, which are illustrated in Figure 9 in a similar way as in Figure 8. Obviously, the increase of φ_k brings enhanced convergence performance to the model training. However, we can see that in *Alibaba* (Figure 9(a)), the convergence curves under different φ_k values are quite divergent, which means that the model training on *Alibaba's* dataset is sensitive to the change of φ_k . In contrast, the convergence curves in *Microsoft* are much more stable and insensitive to the change of φ_k . Such differences across cloud providers are primarily due to their distinct data distributions, highlighting the need for provider-specific training strategies.

Further analysis of Figure 8(b) reveals that RMSE curves under "mid" and "low" confidence categories oscillate more significantly than that under "high" confidence category. Similarly, in Figure 9(a), when $\varphi_k = 4$, the RMSE curve in *Alibaba* also oscillates significantly. This is primarily due to inherent difference between the synthesized data and local real data, and the heterogeneity across different cloud providers' workload datasets. With the increase of synthesized data samples that follow the patterns of other cloud providers or degradation of synthesized data quality, the model parameter update direction differs from the training purely conducted on local real data samples. The static learning rate is probably sub-optimal under these cases, causing the model training to repeatedly overshoot the optimal solution. Therefore, the learning rate needs adjustment according to the quality and amount of synthesized samples to maintain model training convergence stability.

Fig. 10: Overall architecture of $P^3Forecast$.

Challenge 3: given that both the quality and amount of synthesized data impact model convergence and the impact varies across cloud providers, how to design the query mechanism and personalized training approach of workload prediction models that adaptively adjusts data augmentation and learning rate according to the quality and amount of synthesized data samples?

4 SYSTEM DESIGN

Suppose the collaborative training of workload prediction models involves K cloud providers (clients), which have highly Non-IID workload datasets. $P^3Forecast$ consists of the following three stages as shown in Figure 10:

1. **Workload Data Synthesis Quality Assessment** (Section 4.1). First, given the local workload time series and a synthesized workload time series, each client calculates the distance matrix, which reflects both the temporal value discrepancy and pattern similarity between the two time series. Then, each client obtains its cumulative distance matrix via dynamic programming as in Equation (1). Finally, by normalizing the minimum cumulative distance with the length of the longer one among the two time series, each client obtains its *pattern-aware DTW* as the data synthesis quality assessment result.

2. **Federated GAN Training based on Data Synthesis Quality** (Section 4.2). By using the *pattern-aware DTW* as the weight of model updates from the cloud providers, the FL server aggregates the generators and discriminators to train the global GAN model, which captures the workload features of all the cloud providers. Upon the completion of FL training, the FL server sends the trained global generator to the clients for workload data synthesis.

3. **Post-Training of Local Workload Prediction Models** (Section 4.3). After receiving the trained generator, each client first uses it to synthesize data samples, and extracts the most informative ones based on a comprehensive evaluation metric and its corresponding query mechanism. Then, each client conducts training using both real and synthesized data samples, where the comprehensive evaluation

metric serves as the weight of the synthesized data samples. Throughout the training process, the data augmentation and learning rate are adaptively adjusted, guided by the prediction performance improvement of the model, to enhance convergence stability and model performance.

4.1 Workload Data Synthesis Quality Assessment

Given the effectiveness of DTW in measuring time series heterogeneity and its deficiencies illustrated in Section 3.5, we aim to make it insusceptible to time series length and reliable for the comparison of temporal patterns. Considering that the first-order difference reflects the volatility of a time series x (defined as $\nabla x_i = x_i - x_{i-1}$) [56], which measures the magnitude of growth or decline at each time point (i.e., change pattern), we incorporate it in DTW for data synthesis quality assessment, which is called the *pattern-aware DTW*.

Specifically, given two time series x and y with the lengths of n and m , respectively. Suppose the first-order differences of two time series x and y at the i -th and the j -th time points are ∇x_i and ∇y_j , respectively. Instead of directly calculating the Euclidean distance between x_i and y_j as the element $d(i, j)$ in the distance matrix D (Section 3.1), we additionally calculate the Euclidean distance between ∇x_i and ∇y_j , and use it as $d(i, j)$ if the patterns of x and y are comparable at the respective time points. By saying comparable, we mean ∇x_i and ∇y_j have identical signs (i.e., $\nabla x_i \nabla y_j > 0$) or their absolute difference is smaller than a threshold (i.e., $|\nabla x_i - \nabla y_j| < \epsilon$). The rationale is that when ∇x_i and ∇y_j are comparable, we prefer D to reflect the similarity between patterns rather than the distance between points in the time series. Thus, in *pattern-aware DTW*, $d(i, j)$ is calculated as:

$$d(i, j) = \begin{cases} |\nabla x_i - \nabla y_j|, & \nabla x_i \nabla y_j > 0 \vee |\nabla x_i - \nabla y_j| < \epsilon \\ |x_i - y_j|, & \text{otherwise.} \end{cases} \quad (3)$$

Note that the threshold ϵ (e.g., 0.001) is set to ensure that ∇x_i and ∇y_j are sufficiently approximate when they have different signs, which can be adjusted according to specific workload data synthesis quality assessment requirements. Then, we obtain the minimum cumulative distance between x and y , i.e., $\tilde{d}(n, m)$, via dynamic programming as in Equation (1). Finally, as illustrated in Section 3.5, to make *pattern-aware DTW* insusceptible to time series length, the distance between x and y is calculated as $DTW_p(x, y) = \frac{\tilde{d}(n, m)}{\max\{n, m\}}$, i.e., unit-length distance. Thus, $DTW_p(x, y)$ reflects both their distance between temporal values and similarity between temporal patterns regardless of their lengths. The workflow of calculating the *pattern-aware DTW* distance is illustrated in “Stage 1” of Figure 10, where the heatmap shows the cumulative distance matrix between the original workload sequence (red) and the synthesized one (blue), which have equal lengths. The element highlighted with the red square is $\tilde{d}(n, m)$. After normalizing it by $\max\{n, m\}$, it represents the *pattern-aware DTW* distance between the two time series.

4.2 Federated GAN Training based on Data Synthesis Quality

According to Section 3.4, we have known that the workload data in different cloud providers is Non-IID. If we directly

utilize the data for FL training of workload prediction models, the obtained models may suffer from severe performance degradation [20], [21]. Instead, we let the clients collaboratively train a GAN model via Federated GAN training as described in Section 3.2, which can be utilized by each client to synthesize its own IID training data.

However, the traditional Federated GAN training methods cannot adequately attend to the temporal features unique to time-series data. Considering that the *pattern-aware* DTW distance between the local workload data of each cloud provider (denoted as $\mathbf{x}_k$) and the data synthesized by the local GAN model at the t -th training round (denoted as $G_k^t(z)$), i.e., $\text{DTW}_p(\mathbf{x}_k, G_k^t(z))$, reflects their discrepancy in temporal features, we propose to use it to weigh the model updates in Federated GAN training. Specifically, as $\text{DTW}_p(\mathbf{x}_k, G_k^t(z))$ actually measures the unit-length distance (i.e., the larger it is, the less similar $\mathbf{x}_k$ is to $G_k^t(z)$), we take its reciprocal $\text{DTW}_p^{-1}(\mathbf{x}_k, G_k^t(z))$ as their similarity. Moreover, as the *pattern-aware* DTW distances of the K cloud providers are measured on different scales and independent of each other, we let the aggregation server normalize them as:

$$\alpha_k = \frac{\text{DTW}_p^{-1}(\mathbf{x}_k, G_k^t(z))}{\sum_{k=1}^K \text{DTW}_p^{-1}(\mathbf{x}_k, G_k^t(z))}. \quad (4)$$

Additionally, to avoid the vanishing gradient problem (i.e., when the discriminator is much more powerful than the generator, the generator always fails to produce convincing data samples [32]), we simultaneously aggregate the local discriminators and generators of the K cloud providers during FL training, ensuring that they have the same power in confronting each other. By following the objective defined in Equation (2), the aggregation of local discriminators and generators at the t -th round can be described as:

$$\begin{cases} D^t \leftarrow \sum_{k=1}^K \alpha_k D_k^t \\ G^t \leftarrow \sum_{k=1}^K \alpha_k G_k^t \end{cases} \quad (5)$$

After aggregation, the global generator G is optimal if and only if it has learned the temporal features of all data samples. That is, the data space of G covers the overall workload data space of all the cloud providers (i.e., $\bar{\mathbf{X}} = \cup_{k=1}^K \mathbf{x}_k$). Note that since the GAN model aggregation step in Equation (5) is independent of model specifics, it can be actually applied to any existing GAN model architectures. The workflow of our proposed Federated GAN training method is illustrated in “Stage 2” of Figure 10.

4.3 Post-Training of Local Workload Prediction Models

In the exploratory experiments detailed in Section 3.6, we observed that even when the global generator is trained on data from all cloud providers, the synthesized data samples differ noticeably from real ones. This limitation impedes clients from achieving rapid convergence towards the optimal direction during the personalized training of workload prediction models. From the client’s perspective, each participant seeks to acquire the most informative synthesized data samples to effectively augment its local training dataset. To realize this goal, we develop a post-training method of local workload prediction models by following the rationale of GAAL [50]–[53]. The method utilizes the trained generator to synthesize data samples

and incorporates a novel query mechanism, which employs a comprehensive evaluation metric to assess the informativeness of synthesized data samples based on the discriminator’s confidence and workload prediction performance. This mechanism can filter out low-quality synthesized data samples and help extract the most informative ones for adaptive training of the workload prediction models.

Specifically, suppose $\mathbf{X}_k^t$ represents the overall training dataset of the k -th cloud provider at the t -th post-training epoch, which consists of the synthesized data queried in previous epochs and the cloud provider’s local real data ($\mathbf{x}_k$). The criterion for querying the most informative synthesized data samples in terms of the proposed comprehensive evaluation metric can be formulated as:

$$\mathbf{Q}_k^t = \left\{ G(z) \mid \begin{array}{l} R_k^t(s)_{s \in G(z)} \geq \text{MEAN}_{s \in \mathbf{X}_k^t} (R_k^t(s)) \\ \text{and } \mathbf{x}_k \subsetneq \mathbf{X}_k^t \text{ or } \mathbf{x}_k = \mathbf{X}_k^t \end{array} \right\}, \quad (6)$$

where $\mathbf{Q}_k^t$ is the synthesized data samples queried at the t -th epoch, $R_k^t(s)_{s \in G(z)}$ is the informativeness score of a data sample (workload time series) $s \in G(z)$ synthesized at the t -th epoch, and $\text{MEAN}_{s \in \mathbf{X}_k^t} (R_k^t(s))$ is the mean score of all the data samples in $\mathbf{X}_k^t$. The criterion means that the queried data samples must either improve the training data quality in terms of informativeness score ($R_k^t(s)_{s \in G(z)} \geq \text{MEAN}_{s \in \mathbf{X}_k^t} (R_k^t(s))$ and $\mathbf{x}_k \subsetneq \mathbf{X}_k^t$), or augment the training dataset for the first time (i.e., $\mathbf{x}_k = \mathbf{X}_k^t$). Specifically, the informativeness score of a given sample $s \in \mathbf{S}$ is defined as:

$$R_k^t(s)_{s \in \mathbf{S}} = \gamma_k \cdot f_d(s) + (1 - \gamma_k) \cdot f_{\text{RMSE}}(s), \quad (7)$$

where $f_d(s) \in [0, 1]$ denotes the confidence score of s output by the discriminator, and $f_{\text{RMSE}}(s)$ is the performance score of the workload prediction model measured in terms of the RMSE on s . $\gamma_k \in [0, 1]$ is used for controlling the weights of these two scores. Considering that a lower RMSE indicates better model performance, $f_{\text{RMSE}}(s)$ is calculated using the following normalization:

$$f_{\text{RMSE}}(s) = \frac{\max_{s' \in \mathbf{S}} \text{RMSE}(s', \hat{s}') - \text{RMSE}(s, \hat{s})}{\max_{s' \in \mathbf{S}} \text{RMSE}(s', \hat{s}') - \min_{s' \in \mathbf{S}} \text{RMSE}(s', \hat{s}')}, \quad (8)$$

where $\hat{s}$ and $\hat{s}'$ are the prediction outputs of the local workload prediction model, with s and s' as the inputs, respectively. After normalization, a lower RMSE on s means better prediction accuracy of the model, and hence corresponds to a larger $f_{\text{RMSE}}(s)$ value. Although $f_d(s)$ helps select more informative synthesized data samples by leveraging the discriminator’s ability to capture cross-cloud workload patterns, it remains static throughout the post-training process. In contrast, along with the convergence of the workload prediction model during post-training, $f_{\text{RMSE}}(s)$ gradually increases. Therefore, we propose to gradually increase the influence of $f_{\text{RMSE}}(s)$ in Equation (7) by linearly decreasing γ_k per epoch with a step size $\Delta\gamma_k$. Thus, $R_k^t(s)_{s \in \mathbf{S}}$ incorporates both the data synthesis quality assessed in terms of discriminator’s confidence score, and model performance improvement assessed in terms of prediction RMSE, providing a comprehensive informativeness evaluation of synthesized data.

In the initialization phase of post-training, each cloud provider begins by querying the first set of synthesized data samples ($\mathbf{Q}_k^1$) that have the same length as that of x_k by

Equation (6), and merges it into the initial training dataset as $\mathbf{X}_k^1 = \mathbf{x}_k \cup \mathbf{Q}_k^1$. Correspondingly, the post-training per epoch consists of model trainings on $\mathbf{x}_k$ and $\mathbf{Q}_k^t$, respectively. A previous work [57] suggests that properly reweighting more informative samples can enhance generalization, and thereby results in better leverage of the augmented data samples. Based on this insight and the aforementioned comprehensive informativeness score, for each cloud provider, we formulate the overall empirical loss minimization objective of post-training as:

$$\min_{\theta_k} \mathbb{E}_{s \sim \mathbf{x}_k} [l_{\text{real}}(s; \theta_k)] + \mathbb{E}_{s_z \sim \mathbf{Q}_k^t} [w_{s_z} \cdot l_{\text{syn}}(s_z; \theta_k)], \quad (9)$$

where θ_k denotes the workload prediction model, $l_{\text{real}}(\cdot)$ denotes the loss term computed on a real data sample $s \sim \mathbf{x}_k$, and $l_{\text{syn}}(\cdot)$ denotes the loss term computed on a synthesized data sample $s_z \sim \mathbf{Q}_k^t$. w_{s_z} denotes the relative informativeness weight of s_z within $\mathbf{Q}_k^t$, and is defined as:

$$w_{s_z} = \frac{R_k^t(s_z)}{\sum_{s'_z \in \mathbf{Q}_k^t} R_k^t(s'_z)}. \quad (10)$$

From Section 3.6, we have known that the impact of the quality and amount of synthesized data samples on model convergence varies across cloud providers. Moreover, along with the incorporation of more synthesized data samples after several epochs, the quality and size of $\mathbf{X}_k^t$ also change over time. Therefore, we design a personalized training strategy of the workload prediction model that adjusts learning rate in accordance with the quality and size of the training dataset. Specifically, after each epoch t , the cloud provider evaluates the performance improvement of the workload prediction model by comparing the RMSEs of prediction results at the t -th epoch (denoted as $\text{RMSE}(\mathbf{X}_k^t, \hat{\mathbf{X}}_k^t)$) and the $(t-1)$ -th epoch (denoted as $\text{RMSE}(\mathbf{X}_k^{t-1}, \hat{\mathbf{X}}_k^{t-1})$), which has two possible cases:

$$\begin{aligned} f_{\text{RMSE}}^{t,t-1}(\mathbf{X}_k^t) &= \text{RMSE}(\mathbf{X}_k^t, \hat{\mathbf{X}}_k^t) - \text{RMSE}(\mathbf{X}_k^{t-1}, \hat{\mathbf{X}}_k^{t-1}) \\ &= \begin{cases} < \sigma_k, & \mathbf{X}_k^t = \mathbf{X}_k^{t-1} \\ \geq \sigma_k, & \mathbf{X}_k^t = \mathbf{X}_k^{t-1} \cup \mathbf{Q}_k^t. \end{cases} \end{aligned} \quad (11)$$

where $\sigma_k \in [0, 1]$ is the threshold reflecting the cloud provider's tolerance to workload prediction performance degradation. For different cloud providers, the optimal settings of $\Delta\gamma_k$ and σ_k may vary due to differences in workload characteristics and tolerance to prediction performance degradation. To this end, we systematically examine different combinations of $\Delta\gamma_k$ and σ_k during personalized post-training, and select the one that achieves the best RMSE per cloud provider, of which details are elaborated in Section 5.3.3. A smaller σ_k corresponds to more timely adjustment of model convergence, but at the cost of more repeated adjustment overhead. If $f_{\text{RMSE}}^{t,t-1}(\mathbf{X}_k^t) < \sigma_k$, it means current learning rate results in steady model convergence, and hence does not need adjustment. The training dataset $\mathbf{X}_k^t$ also remains unchanged. Otherwise, the prediction model performance is not improving, which triggers the synthesis and query of new data samples $\mathbf{Q}_k^t$ by Equation (6), and its following merging with the existing training dataset as $\mathbf{X}_k^t = \mathbf{X}_k^{t-1} \cup \mathbf{Q}_k^t$. Meanwhile, we propose to adjust the learning rate in accordance with $\mathbf{X}_k^t$ as well.

On the one hand, inspired by the observations in Section

Algorithm 1: Workflow of $P^3\text{Forecast}$.

1 **Server executes:**

2 Initialize the global GAN model D^0 and G^0 ;

3 **for each round** $t = 1, 2, \dots$ **do**

4 **for** $k = 1$ **to** K **in parallel do**

5 $D_k^t, G_k^t, \text{DTW}_p(\mathbf{x}_k, G_k^t(z)) =$
 ClientUpdate(D^{t-1}, G^{t-1});

6 **for** $k = 1$ **to** K **do**

7 $\alpha_k = \frac{\text{DTW}_p^{-1}(\mathbf{x}_k, G_k^t(z))}{\sum_{k=1}^K \text{DTW}_p^{-1}(\mathbf{x}_k, G_k^t(z))}$;

8 Aggregate local GAN models by Equation (5);

9 **for** $k = 1$ **to** K **in parallel do**

10 ClientPostTraining(G);

11 **Clients execute:**

12 Initialize local prediction model;

13 ClientUpdate(D^{t-1}, G^{t-1}):

14 Set $D_k^{t-1} = D^{t-1}, G_k^{t-1} = G^{t-1}$;

15 **for each epoch** $e = 1, 2, \dots$ **do**

16 $D_k^t, G_k^t \leftarrow$ Update local GAN model with $\mathbf{x}_k$;

17 Synthesize workload data $G_k^t(z)$;

18 Calculate $\text{DTW}_p(\mathbf{x}_k, G_k^t(z))$ as in Section 4.1;

19 **return** $D_k^t, G_k^t, \text{DTW}_p(\mathbf{x}_k, G_k^t(z))$;

20 ClientPostTraining(G):

21 Initialize synthesized data samples $\mathbf{Q}_k$;

22 $\mathbf{X}_k^1 \leftarrow \mathbf{x}_k \cup \mathbf{Q}_k^1$;

23 **for each epoch** $t = 1, 2, \dots$ **do**

24 Train model θ_k on $\mathbf{X}_k^t$ with Equation (9);

25 Update $\mathbf{X}_k^t$ by Equation (11);

26 Update η_k by Equation (12);

3.6, we expect the learning rate to gradually decrease along with the increase of the training dataset size, in order to maintain model convergence stability. On the other hand, we don't expect the learning rate to decrease too fast, as an excessively low learning rate could lead to model under-fitting. Therefore, given the ratio between the amount of synthesized data samples and the amount of local data samples of the k -th cloud provider (i.e., φ_k), we use an exponential function of φ_k to scale the learning rate η_k as:

$$\eta_k = \eta^0 e^{-\varphi_k}, \quad (12)$$

where η^0 is the initial learning rate. Notably, the value of φ_k is dynamically updated in response to the scale of $\mathbf{X}_k^t$ throughout the execution process, requiring no manual configuration. Finally, the post-training process of local workload prediction models is illustrated in "Stage 3" of Figure 10.

Cloud workloads inherently evolve over time, causing the underlying data distribution and temporal patterns to change. Such non-stationary characteristic may lead to concept drift, where a GAN model trained on historical data becomes less effective on future workloads. In such cases, if the error caused by concept drift exceeds the tolerance threshold of any participating cloud providers, they can initiate another round of federated GAN training to refresh the informativeness of their synthesized data.

4.4 Summary

The detailed workflow of $P^3\text{Forecast}$ can be summarized as Algorithm 1. At Lines 2 and 12, the server and clients ini-

tialize the global GAN model and local prediction models, respectively. At Lines 4-5 and 13-19, each client updates its GAN model with its original workload data $\mathbf{x}_k$, synthesizes data $G_k^t(z)$, and computes the *pattern-aware* DTW distance $\text{DTW}_p(\mathbf{x}_k, G_k^t(z))$ as in Section 4.1. At Lines 6-8, the server aggregates the clients' local GAN models as in Section 4.2. At Lines 9-10 and 20-27, each client uses the trained global generator to augment its training dataset and post-train its workload prediction model as in Section 4.3.

It is worth noticing that unlike most existing FL personalization methods that require consistent model architectures across clients, $P^3\text{Forecast}$ grants clients high flexibility in independently specifying their own prediction model architectures (e.g., LSTM, CNNs, Transformers, etc.) according to their own needs, which is especially suitable for cloud providers with heterogeneous model preferences.

4.5 Security Analysis

The previous methods using shared GANs for FL training data augmentation [23], [24], [29] require participants to share the base noise that covers all the heterogeneous data features across clients, which may cause privacy leakage and violate the privacy regulations of most cloud providers [14], [15]. In contrast, the clients in $P^3\text{Forecast}$ only upload GAN model parameters and the data synthesis quality assessment result DTW_p to the FL server for normalization and model aggregation. For the former, $P^3\text{Forecast}$ can refer to various privacy preservation schemes (e.g., HE, DP) to prevent adversaries from accessing the actual model parameters. For the latter, DTW_p is a scalar value and does not contain any privacy-sensitive information. Therefore, $P^3\text{Forecast}$ can ensure the privacy protection of clients while allowing the collaborative training of GAN models for IID training data augmentation, and personalization of local workload prediction models.

5 PERFORMANCE EVALUATION

5.1 Comparison Methods

To better demonstrate the contribution of different components, we divide $P^3\text{Forecast}$ into two forms: $P^3\text{Forecast}_P$, which only takes into account prediction performance in the data query mechanism of post-training (i.e., the original conference version [38]), and $P^3\text{Forecast}_{Q+P}$, which takes into account both data synthesis quality and prediction performance in the data query mechanism of post-training (i.e., the extended journal version).

For workload prediction evaluation, we compare $P^3\text{Forecast}_P$ and $P^3\text{Forecast}_{Q+P}$ with a GRU-based cloud workload prediction model [8] (denoted as *GRU*), a novel Multi-Layer-Perceptron-based (MLP-based) cloud workload prediction model [10] (denoted as *WPMixer*), the standard FL framework [18] (denoted as *FedAvg*), a representative FL framework for tackling heterogeneity [22] (denoted as *FedProx*), and a personalized FL framework for addressing heterogeneity [27] (denoted as *pFedMe*). Specifically, *GRU* exploits a sliding window and a revised GRU neural network for cloud workload prediction. *WPMixer* utilizes patching, multi-resolution wavelet decomposition, and mixing for workload prediction. Note that *GRU* and *WPMixer*

train workload prediction models per cloud provider via solely using its local data. *FedAvg* weighs and aggregates the clients' uploaded model parameters by data quantity. *FedProx* introduces a proximal term to the objective function of *FedAvg* to penalize the deviation between local models and the global model. *pFedMe* leverages Moreau envelopes to decouple client personalization from global model learning via a bi-level FL framework.

For convergence analysis, we compare $P^3\text{Forecast}_P$ and $P^3\text{Forecast}_{Q+P}$ with a GAN variant (denoted as *IFL-GAN*) [34] and the vanilla FL GAN (denoted as *Baseline*) [30]. Specifically, *IFL-GAN* aims to train a globally shared GAN model via aggregating the generators' model updates trained on the clients' local data, of which data synthesis quality is assessed in terms of the Maximum Mean Discrepancy (MMD). In *Baseline*, the GAN models are directly aggregated by federated averaging, of which weights are determined by the amount of data samples per client.

To illustrate the effectiveness of our proposed *pattern-aware* DTW distance, we also evaluate the convergence performance of a variant of $P^3\text{Forecast}_{Q+P}$, which follows the same design as $P^3\text{Forecast}_{Q+P}$, but uses the DTW distance in FL model aggregation (denoted as $P^3\text{Forecast-DTW}$).

5.2 Experimental Settings

Datasets. The experiments are driven by the datasets introduced in Section 3. Specifically, we use the first 70% as the local training dataset per cloud provider. Since there is no available cross-cloud workload dataset covering various types of workload patterns, we extract the remaining 30% workload data from each cloud provider to build the *combined testing dataset*. In addition, to highlight the advantages brought by FL, we also use the remaining 30% workload data of each cloud provider as the *local testing dataset* per cloud provider.

Models. For fairness, we select the TimeGAN model architecture [54] for all the comparison methods that use GAN for workload data synthesis. Specifically, both the embedding network and the generator use a GRU network consisting of one input layer with the size of 2, three hidden layers with 256 hidden units and one output layer with the size of 256. The recovery network uses a GRU network consisting of one input layer with the size of 256, three hidden layers with 256 hidden units and one output layer with the size of 2. The discriminator uses a GRU network consisting of one input layer with the size of 256, three hidden layers with 256 hidden units and one output layer with the size of 1. Moreover, considering that the comparison methods do not grant clients the flexibility of specifying their own workload prediction models as in $P^3\text{Forecast}$, we let all cloud providers use the same GRU network as the prediction model for fair evaluation, which consists of one input layer with the size of 2, one hidden layer with 512 hidden units, and one output layer with the size of 2.

Implementation. We conduct evaluations on a Linux server with 2 GeForce RTX 3090 GPUs, 1 Intel Xeon CPU, and 256 GB memory. We implement our framework with PyTorch. The FL clients are *Alibaba*, *Microsoft*, *Google*, *Alibaba-AI*, *HPC-KS*, *HPC-HF* and *HPC-WZ*.

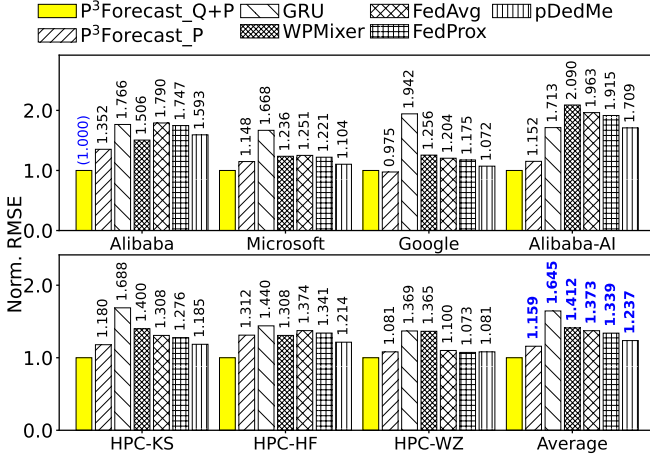


Fig. 11: Workload prediction accuracy on different cloud providers using the combined testing dataset (normalized by the RMSE of $P^3Forecast_Q+P$).

5.3 Experimental Results

5.3.1 Workload Prediction Accuracy

In this section, we evaluate the performance of different methods in terms of workload prediction accuracy. Specifically, in *GRU* and *WPMixer*, the cloud providers separately train their prediction models with their isolated workload data for 30 epochs. In *FedAvg*, *FedProx* and *pFedMe*, the global FL training is conducted for 6 rounds with 5 local training epochs per round. The hyper-parameters of these comparison methods are fine-tuned to reflect their optimal performance. In $P^3Forecast_P$ and $P^3Forecast_Q+P$, the global FL training of the TimeGAN model is conducted for 10 rounds with 500 local training epochs per round. The post-training of the local prediction model is conducted for 30 rounds with the workload data synthesized by the trained TimeGAN model. In all the methods, the batch sizes for the training of the GRU and TimeGAN models are 128, while the lengths of observation time step for the two models are 64 and 65, respectively. In the TimeGAN model, the first 64 time steps are used as the input and the last 1 time step is used as the output. We employ the Adam optimizer [58] for both the GRU and TimeGAN models, using an initial learning rate of 0.001, and beta values of 0.9 and 0.999, respectively. As for post-training, we initialize $\gamma_k = 1$, and set $\Delta\gamma_k = 0.05$, $\sigma_k = 0.005$ for all the cloud providers in this experiment.

For each cloud provider, we measure the average RMSEs of workload prediction results on CPU utilization and memory utilization in different methods, which are illustrated in Figure 11. For the convenience of comparison, all RMSEs are normalized by the RMSE of $P^3Forecast_Q+P$. The overall performance across all the cloud providers generally follows: $P^3Forecast_Q+P < P^3Forecast_P < pFedMe < FedProx < FedAvg < WPMixer < GRU$. We can observe that all the FL-based methods (i.e., *FedAvg*, *FedProx*, *pFedMe*, $P^3Forecast_P$ and $P^3Forecast_Q+P$) outperform *GRU* and *WPMixer* in almost all the cloud providers. This is primarily because that in *GRU* and *WPMixer*, the prediction models are separately trained per cloud provider with its isolated workload dataset. Despite utilizing multi-resolution wavelet decomposition combined with patching and mixing to capture intricate features across both time and frequency domains,

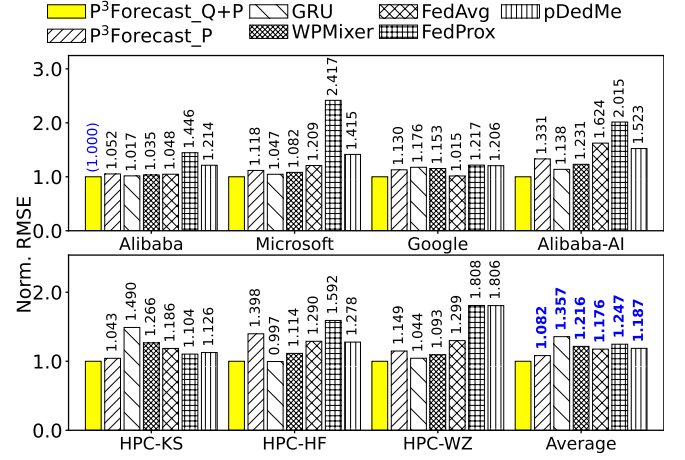


Fig. 12: Workload prediction accuracy on different cloud providers using the local testing dataset (normalized by the RMSE of $P^3Forecast_Q+P$).

WPMixer remains limited by its reliance on isolated datasets. Thus, on the combined testing dataset covering various workload patterns across the cloud providers, the trained models of most cloud providers cannot achieve comparable performance as the ones collaboratively trained with the FL algorithms. Moreover, we can notice that the performance of *GRU* is especially bad on *Google*, which is due to the singular workload patterns of *Google* compared to the other cloud providers. The prediction model solely trained on *Google*'s data has insufficient knowledge on the workload patterns of the other cloud providers, and hence result in the model's inferior performance on the testing dataset.

We can also observe that the RMSEs of workload prediction results in *FedAvg* and *FedProx* are quite approximate. This is mostly because that in these methods, the prediction models trained via FL are not personalized according to the heterogeneity degree of local data. For the cloud providers that already have a rich variety of workload patterns, the patterns learned from the others may actually deteriorate their model performance (e.g., *Alibaba-AI*). In contrast, *pFedMe* incorporates a personalization mechanism that partially alleviates this issue. However, its reliance on Moreau envelope-based regularized loss functions to constrain model parameters remains insufficient to address the heterogeneity in cloud workload prediction scenarios.

In contrast, it is evident that $P^3Forecast_P$ and $P^3Forecast_Q+P$ outperform the other methods, especially for $P^3Forecast_Q+P$, which is 15.9%, 23.7%, 33.9%, 37.3%, 41.2% and 64.5% more accurate in average over all the cloud providers than $P^3Forecast_P$, *pFedMe*, *FedProx*, *FedAvg*, *WPMixer*, and *GRU*, respectively. For the cloud providers with relatively more singular workload patterns, (e.g., *Google*), $P^3Forecast_Q+P$ can achieve as much as 94.2% improvement in accuracy compared to *GRU*. The significant performance improvement can be attributed to our proposed Federated GAN training method based on workload data synthesis quality assessment, which ensures the capture of cross-cloud workload patterns in data synthesis, and post-training of workload prediction models, which ensures the personalized augmentation of training data according to the heterogeneity degree of cloud providers' local data. As a result, $P^3Forecast_Q+P$ can effectively mitigate data hetero-

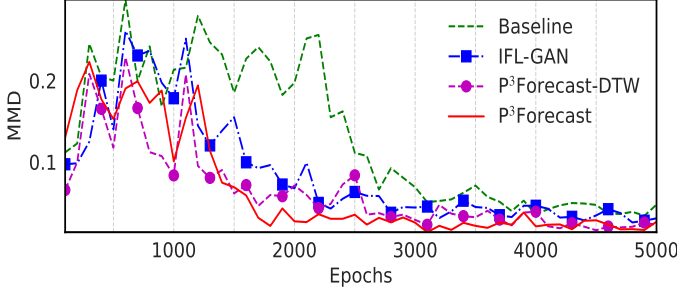


Fig. 13: Convergence performance of different methods.

generality while enjoying the workload patterns contributed across the clouds. Note that since Federated GAN training is parallel, while workload prediction model post-training is local, $P^3\text{Forecast_Q+P}$ scales effectively, as seen in previous Federated GANs [30], [32].

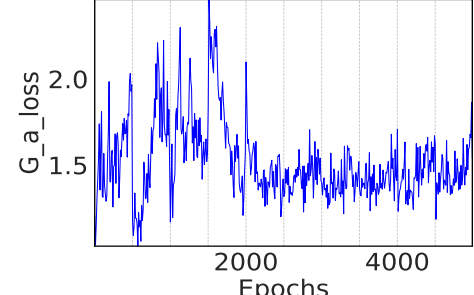
We further conduct the same experiments on the local testing dataset and present the results in Figure 12. We can observe that $P^3\text{Forecast_Q+P}$ outperforms $P^3\text{Forecast_P}$ on the majority of cloud providers. On average, $P^3\text{Forecast_Q+P}$ is 8.2%, 17.6%, 18.7%, 24.7% and 35.7% more accurate than $P^3\text{Forecast_P}$, FedAvg , $p\text{FedMe}$, WPMixer , FedProx and GRU . This result demonstrates that even if certain cloud providers have no interest in learning universal workloads covering various patterns, and solely focus on their local workloads, they can still benefit from the workload prediction model trained with $P^3\text{Forecast}$. In summary, these results confirm that the personalized workload prediction model training framework of $P^3\text{Forecast}$ is universally effective, enabling better prediction performance on both heterogeneous and local workload data across cloud providers.

5.3.2 Convergence Analysis of Federated GAN Training

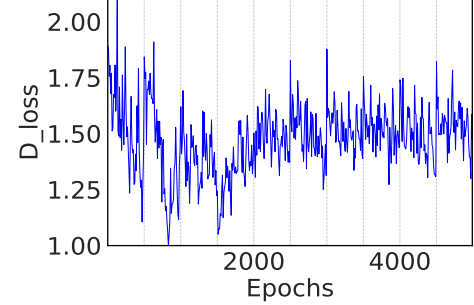
In this section, we evaluate the convergence performance, as well as the data synthesis quality, of the methods. We use the MMD as the metric of data synthesis quality. The change of the MMD along with the training progress can effectively reflect how well the methods converge [59]–[61]. Specifically, the MMD between the local data x and the synthesized data $G(z)$ is calculated as:

$$\text{MMD} = \sup \left\| \mathbb{E}_{x \sim p(x_k)} [\phi(x)] - \mathbb{E}_{z \sim p(z)} [\phi(G(z))] \right\|^2, \quad (13)$$

where $\phi(\cdot)$ is the Gaussian kernel function. A lower MMD indicates a better data synthesis quality. During the training of the Federated GAN models in each method, we record the MMD between x and $G(z)$ every 100 epochs, which is illustrated in Figure 13. We can see that in the first 1300 epochs, IFL-GAN , $P^3\text{Forecast}$ and $P^3\text{Forecast-DTW}$ converge conspicuously faster than Baseline , which confirms the effectiveness of taking into account data synthesis quality in the weighing and aggregation of model updates in Federated GAN training. However, after 1300 epochs, $P^3\text{Forecast}$ and $P^3\text{Forecast-DTW}$ converge faster and finally stabilize at better MMD scores than IFL-GAN . This is primarily because that although MMD is effective in measuring the dissimilarity between sample distributions, it neglects the temporal features unique to time-series data. Furthermore, we also notice that $P^3\text{Forecast}$ quickly stabilizes at around 1800 epochs, while $P^3\text{Forecast-DTW}$ has to spend >4000 epochs to achieve a comparable MMD level. This is primarily because our proposed *pattern-aware DTW* enables the GAN model



(a) The adversarial generator loss.



(b) The discriminator loss.

Fig. 14: The losses of the GAN model in $P^3\text{Forecast}$.

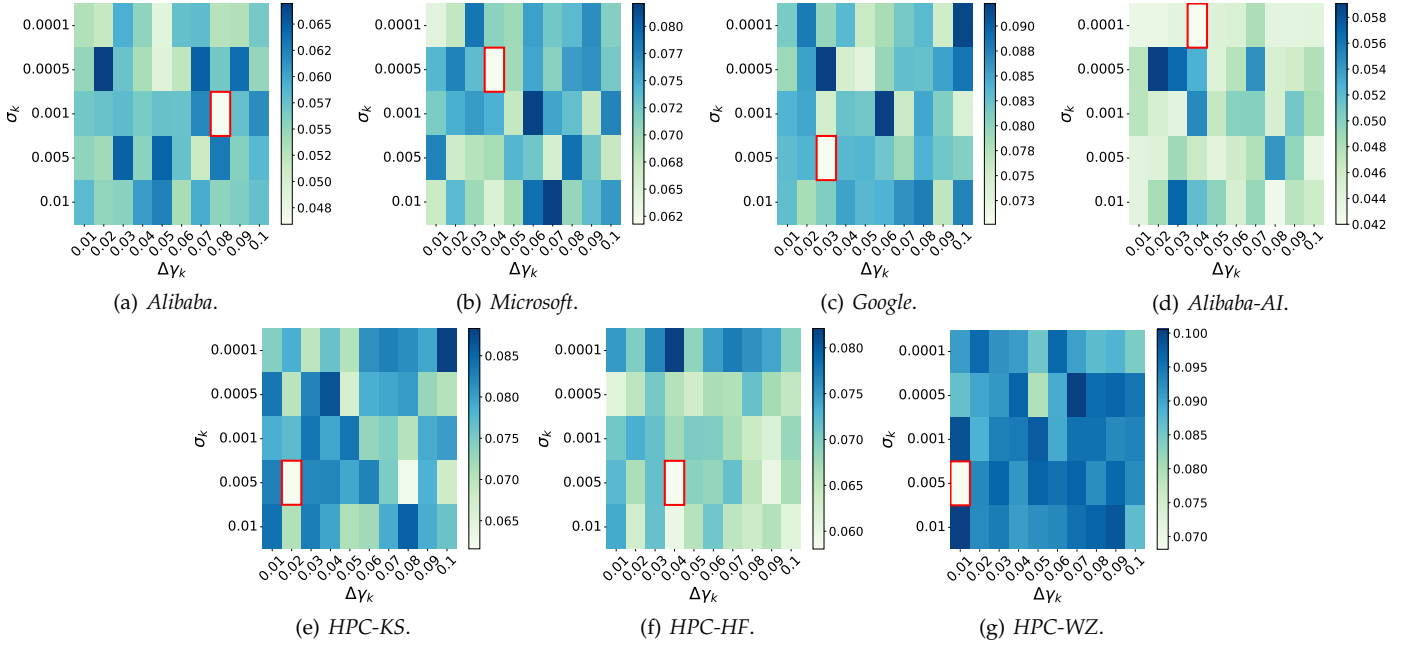
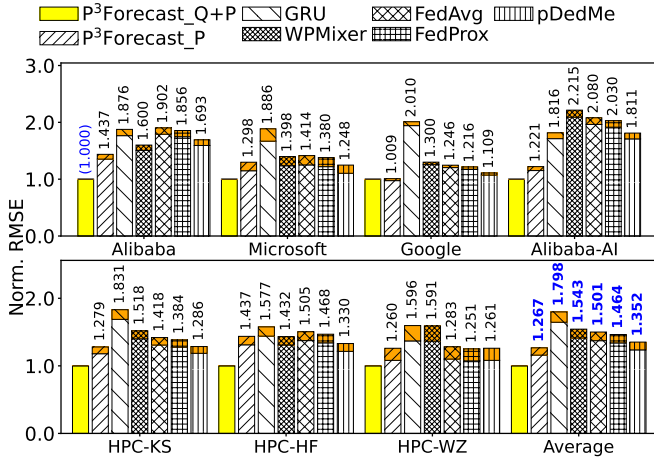
to synthesize data that is similar to the real one not only in temporal distance, but also in patterns.

Moreover, we also record the adversarial generator loss and the discriminator loss of the GAN model along with the training progress, which are shown in Figure 14(a) and Figure 14(b), respectively. We can observe that the adversarial generator loss and the discriminator loss generally exhibit opposite trends: along with the progress of training, the adversarial generator loss initially rises and then falls, while the discriminator loss first falls and then rises. Ultimately, both losses tend to converge, indicating the GAN model has reached or approached the Nash equilibrium.

5.3.3 Parameter Sensitivity Analysis

Since $\Delta\gamma_k$ directly affects the informativeness score defined in Equation (7), while σ_k specifies each cloud provider's tolerance to workload prediction performance degradation, we further conduct a comprehensive analysis of how $\Delta\gamma_k$ and σ_k influence the post-training performance of each cloud provider. Specifically, except for $\Delta\gamma_k$ and σ_k , all the settings are the same as in Section 5.3.1. We initialize $\gamma_k = 1$ for each cloud provider, and let it decrease by varying $\Delta\gamma_k$ values ranging from 0.01 to 0.09 with a step size of 0.01 throughout training. We vary σ_k over $\{0.0001, 0.0005, 0.001, 0.005, 0.01\}$, which covers a broad yet practically meaningful range of tolerance levels. Then, we measure the RMSEs of prediction results, which are shown in Figure 15. For each cloud provider, the optimal combinations of $\Delta\gamma_k$ and σ_k , which yield the lowest RMSE values, are highlighted with red boxes. As shown in Figure 15, the grid search on $\Delta\gamma_k$ and σ_k exhibits no clear monotonic or structural trends, likely due to the personalized and adaptive characteristics of $P^3\text{Forecast}$'s post-training process. Therefore, the selected parameter combination should be viewed as a robust empirical setting rather than a global optimum with strict boundaries.

To evaluate the benefit of parameter optimization in the post-training phase, we further compare the workload

Fig. 15: Impact of $\Delta\gamma_k$ and σ_k on algorithm performance.Fig. 16: Workload prediction accuracy using the optimal combination of $\Delta\gamma_k$ and σ_k per cloud provider.

prediction accuracy of $P^3Forecast_Q+P$, configured with the provider-specific optimal combination of $\Delta\gamma_k$ and σ_k , with that of the competing methods. Figure 16 illustrates the prediction RMSEs of all the cloud providers, which are normalized by those of $P^3Forecast_Q+P$. Compared with Figure 11, the improvements harvested from using the optimal combinations of $\Delta\gamma_k$ and σ_k are marked with orange segments. We can see that after adjusting $\Delta\gamma_k$ and σ_k , $P^3Forecast_Q+P$ is 26.7%, 35.2%, 46.4%, 50.1%, 54.3% and 79.8% more accurate in average over all the cloud providers when compared with $P^3Forecast_P$, $pFedMe$, $FedProx$, $FedAvg$, $WPMixer$ and GRU , respectively.

5.3.4 Runtime Performance Analysis

We compare the running time of $P^3Forecast_Q+P$ and $P^3Forecast_P$ against that of the other baselines. Notably, GRU and $WPMixer$ do not involve FL or post-training phases. Given that local computations across different cloud providers can be executed in parallel, their total running time is determined by the maximum local execution time. Furthermore, since the post-training phase is unique to

TABLE 1: Comparison of training time.

Method	Training time		
	FL training	Post-training	Sum
$P^3Forecast_P$	27m17s	1m40s	28m57s
$P^3Forecast_Q+P$	27m17s	44s	28m1s
GRU	15s		
$WPMixer$	51s		
$FedAvg$	52s	0s	52s
$FedProx$	60s	0s	60s
$pFedMe$	7m37s	0s	7m37s

$P^3Forecast_Q+P$ and $P^3Forecast_P$, the post-training duration of all the other FL frameworks is recorded as 0 seconds.

As shown in Table 1, $P^3Forecast_Q+P$ consumes less time than $P^3Forecast_P$ in the post-training phase, indicating that our improvements enhance both performance and efficiency. Moreover, training the federated GAN accounts for the majority of the time for both $P^3Forecast_Q+P$ and $P^3Forecast_P$, resulting in higher overall time consumption compared to the baselines. Nevertheless, this extra computational cost is justified by three key advantages. First, as demonstrated in Section 5.3.1, $P^3Forecast_Q+P$ achieves a substantially lower prediction RMSE compared to the other methods, indicating that the additional overhead translates into a clear gain in prediction accuracy. Second, once trained, the federated GAN can continuously generate high-quality, informative synthesized data on demand, which can also be leveraged for other downstream tasks. Third, unlike existing FL-based methods that may suffer from substantial performance degradation in non-IID settings, $P^3Forecast_Q+P$ demonstrates superior adaptability and robustness to heterogeneous client data.

5.3.5 Ablation Analysis

We conduct further ablation analysis to evaluate the effectiveness of different components in the post-training of the local workload prediction models. Specifically, we implement a variant of $P^3Forecast_Q+P$ by removing the

TABLE 2: RMSEs of prediction results in ablation analysis.

Cloud Provider	$P^3Forecast_Q+P$	$P^3Forecast_q$	$P^3Forecast_s$
Alibaba	0.0459	0.0540	0.0848
Microsoft	0.0617	0.0627	0.0675
Google	0.0701	0.0772	0.0942
Alibaba-AI	0.0420	0.0472	0.0449
HPC-KS	0.0615	0.0755	0.0850
HPC-HF	0.0580	0.0798	0.0774
HPC-WZ	0.0681	0.1002	0.1028
Average	0.0582	0.0709	0.0795

learning rate adjustment component (denoted as $P^3Forecast_q$), which means that the model is post-trained by solely using the query mechanism based on the comprehensive informativeness evaluation metric (Equation (6)). Meanwhile, we implement another variant of $P^3Forecast_Q+P$ by completely removing the adaptive post-training stage (denoted as $P^3Forecast_s$), which means the model is trained with static amount of synthesized data samples (i.e., $|x_k|$) and a static learning rate. We measure the RMSEs of workload prediction results per cloud provider in these variants, which are shown in Table 2. For reference, the prediction RMSEs of $P^3Forecast_Q+P$ collected under the optimal settings are also included.

It can be observed that $P^3Forecast_Q+P$ significantly outperforms the others on most cloud providers except for *Microsoft*. The results demonstrate that the personalized training strategy that adaptively adjusts learning rate in accordance with the quality and size of the training dataset is pivotal in improving the model post-training performance of $P^3Forecast_Q+P$. Moreover, we can also observe that $P^3Forecast_s$ underperforms $P^3Forecast_q$ on most cloud providers except for *Alibaba-AI* and *HPC-HF*, and is substantially worse than $P^3Forecast_Q+P$ on all the cloud providers. These results confirm the necessity of the proposed post-training method in personalizing the optimal workload prediction model for cloud providers.

Furthermore, we delve into the model convergence process of the three methods in *Alibaba-AI* and *HPC-WZ*. The changes of RMSE along with the training progress in the two cloud providers are illustrated in Figure 17(a) and Figure 17(b), respectively. In Figure 17(a), $P^3Forecast_Q+P$ achieves faster and more stable convergence than the others. The conspicuous oscillation of convergence in $P^3Forecast_q$ and $P^3Forecast_s$ are mostly due to the absence of the adaptive learning rate strategy, causing the model training to repeatedly overshoot the optimal solution. However, in Figure 17(b), all three methods exhibit varying degrees of convergence oscillation, which is likely due to the inherent inherent difference between the synthesized data and local real data. Despite this challenging condition, $P^3Forecast_Q+P$ consistently converges to a model state with superior performance. In contrast, $P^3Forecast_q$ and $P^3Forecast_s$ cannot match the same model performance by the end of training, further demonstrating the necessity of adaptive learning rate adjustment in stabilizing post-training convergence.

6 CONCLUSION

In this paper, we have extensively demonstrated the high Non-IID nature of cloud workloads and the deficiencies

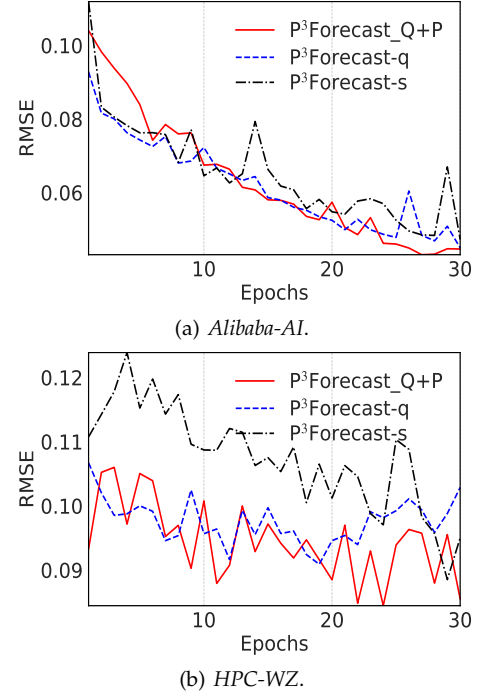


Fig. 17: Comparison of model convergence in *Alibaba-AI* and *HPC-WZ*. of existing methods in realizing the collaborative training of workload prediction models. To address the issues, we proposed $P^3Forecast$, a personalized privacy-preserving cloud workload prediction framework that extends DTW for data synthesis quality assessment, adopts the FL training of a GAN model for training data augmentation across cloud providers, and personalizes the workload prediction model per cloud provider via adaptive and stable post-training. Our experiments conducted on real-world workloads showed that compared with the state-of-the-art, $P^3Forecast$ drastically improved workload prediction accuracy by 23.7%-64.5% in average over all cloud providers, while ensuring the fastest convergence in Federated GAN training. In the future, we plan to further investigate implementation-level details of cloud applications to improve workload prediction accuracy while reducing runtime overhead.

REFERENCES

- [1] J. Kumar and A. K. Singh, "Workload prediction in cloud using artificial neural network and adaptive differential evolution," *Future Generation Computer Systems*, vol. 81, 2018.
- [2] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, "Analysis of large-scale multi-tenant gpu clusters for dnn training workloads," in *Proc. of USENIX ATC*, 2019.
- [3] Q. Weng, W. Xiao, Y. Yu, W. Wang, C. Wang, J. He, Y. Li, L. Zhang, W. Lin, and Y. Ding, "MLaaS in the wild: Workload analysis and scheduling in Large-Scale heterogeneous GPU clusters," in *Proc. of NSDI*, 2022.
- [4] R. N. Calheiros, E. Masoumi, R. Ranjan, and R. Buyya, "Workload prediction using arima model and its impact on cloud applications' QoS," *IEEE TCC*, vol. 3, no. 4, 2015.
- [5] I. K. Kim, W. Wang, Y. Qi, and M. Humphrey, "Forecasting cloud application workloads with cloudinsight for predictive resource management," *IEEE TCC*, vol. 10, no. 3, 2020.
- [6] S. Ouham, Y. Hadi, and A. Ullah, "An efficient forecasting approach for resource utilization in cloud data center using cnn-lstm model," *Neural Computing and Applications*, vol. 33, 2021.

- [7] Z. Chen, J. Hu, G. Min, A. Y. Zomaya, and T. El-Ghazawi, "Towards accurate prediction for high-dimensional and highly-variable cloud workloads with deep learning," *IEEE TPDS*, vol. 31, no. 4, 2019.
- [8] M. Xu, C. Song, H. Wu, S. S. Gill, K. Ye, and C. Xu, "esdnn: deep neural network based multivariate workload prediction in cloud computing environments," *ACM TOIT*, vol. 22, no. 3, 2022.
- [9] F. Zhao, W. Lin, S. Lin, S. Tang, and K. Li, "Mscnet: Multi-scale network with convolutions for long-term cloud workload prediction," *IEEE Transactions on Services Computing*, vol. 18, no. 2, 2025.
- [10] M. M. N. Murad, M. Aktukmak, and Y. Yilmaz, "Wpmmixer: Efficient multi-resolution mixing for long-term time series forecasting," in *Proc. of AAAI*, 2025.
- [11] C. Ruan, Y. Chen, D. Tian, Y. Shi, Y. Wu, J. Li, and C. Li, "Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic LLM workloads," in *Proc. of NSDI*, 2026.
- [12] R. Gong, S. Bai, S. Wu, Y. Fan, Z. Wang, X. Li, H. Yang, and X. Liu, "Past-future scheduler for llm serving under sla guarantees," in *Proc. of ASPLOS*, 2025.
- [13] Y. Xiang, X. Li, K. Qian, W. Yu, E. Zhai, and X. Jin, "Servergen: Workload characterization and generation of large language model serving in production," in *Proc. of NSDI*, 2026.
- [14] Z. Yang, Z. Wu, M. Luo, W.-L. Chiang, R. Bhardwaj, W. Kwon, S. Zhuang, F. S. Luan, G. Mittal, S. Shenker, and I. Stoica, "SkyPilot: An intercloud broker for sky computing," in *Proc. of NSDI*, 2023.
- [15] I. Stoica and S. Shenker, "From cloud computing to sky computing," in *Proc. of HotOS*, 2021.
- [16] O. Gupta and R. Raskar, "Distributed learning of deep neural network over multiple agents," *Journal of Network and Computer Applications*, vol. 116, 2018.
- [17] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *Proc. of S&P*, 2019.
- [18] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. of AISTATS*, 2017.
- [19] P. Mell and T. Grance, *The NIST Definition of Cloud Computing*. National Institute of Standards and Technology, 2011.
- [20] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of fedavg on non-iid data," in *Proc. of ICLR*, 2020.
- [21] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. of ICML*, 2020.
- [22] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. of MLSys*, 2020.
- [23] Z. Zhu, J. Hong, and J. Zhou, "Data-free knowledge distillation for heterogeneous federated learning," in *Proc. of ICML*, 2021.
- [24] Z. Tang, Y. Zhang, S. Shi, X. He, B. Han, and X. Chu, "Virtual homogeneity learning: Defending against data heterogeneity in federated learning," in *Proc. of ICML*, 2022.
- [25] R. Zhou, J. Yu, R. Wang, B. Li, J. Jiang, and L. Wu, "A reinforcement learning approach for minimizing job completion time in clustered federated learning," in *Proc. of INFOCOM*, 2023.
- [26] J. Wolfrath, N. Sree Kumar, D. Kumar, Y. Wang, and A. Chandra, "Haccs: Heterogeneity-aware clustered client selection for accelerated federated learning," in *Proc. of IPDPS*, 2022.
- [27] C. T. Dinh, N. Tran, and J. Nguyen, "Personalized federated learning with moreau envelopes," in *Proc. of NIPS*, 2020.
- [28] S. Wu, Y. Jia, B. Liu, H. Xiang, X. Xu, and W. Dou, "Pfedcs: A personalized federated learning method for enhancing collaboration among similar classifiers," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 20, 2025.
- [29] E. Jeong, S. Oh, H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Communication-efficient on-device machine learning: Federated distillation and augmentation under non-iid private data," *arXiv preprint arXiv:1811.11479*, 2018.
- [30] C. Hardy, E. Le Merrer, and B. Sericola, "Md-gan: Multi-discriminator generative adversarial networks for distributed datasets," in *Proc. of IPDPS*, 2019.
- [31] M. Rasouli, T. Sun, and R. Rajagopal, "Fedgan: Federated generative adversarial networks for distributed data," *arXiv preprint arXiv:2006.07228*, 2020.
- [32] R. Guerraoui, A. Guirguis, A.-M. Kermarrec, and E. L. Merrer, "Fegan: Scaling distributed gans," in *Proc. of Middleware*, 2020.
- [33] J. Zhang, L. Zhao, K. Yu, G. Min, A. Y. Al-Dubai, and A. Y. Zomaya, "A novel federated learning scheme for generative adversarial networks," *IEEE TMC*, vol. 23, no. 5, 2024.
- [34] W. Li, J. Chen, Z. Wang, Z. Shen, C. Ma, and X. Cui, "If-gan: Improved federated learning generative adversarial network with maximum mean discrepancy model aggregation," *IEEE TNNLS*, vol. 34, no. 12, 2023.
- [35] Z. Ma, Y. Liu, Y. Miao, G. Xu, X. Liu, J. Ma, and R. H. Deng, "Flgan: Gan-based unbiased federated learning under non-iid settings," *IEEE TKDE*, vol. 36, no. 4, 2024.
- [36] E. Brophy, "Synthesis of dependent multichannel ecg using generative adversarial networks," in *Proc. of CIKM*, 2020.
- [37] E. Brophy, Z. Wang, Q. She, and T. Ward, "Generative adversarial networks in time series: A systematic literature review," *ACM Computing Surveys*, vol. 55, no. 10, 2023.
- [38] Y. Kuang, L. Yan, and Z. Li, "P3forecast: Personalized privacy-preserving cloud workload prediction based on federated generative adversarial networks," in *Proc. of IPDPS*, 2025.
- [39] "Alibaba cluster data v2018," <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018>, 2023, accessed in October, 2024.
- [40] "Microsoft cluster data," <https://github.com/msr-fiddle/philly-traces>, 2023, accessed in October, 2024.
- [41] "Google cluster data," <https://github.com/google/cluster-data/blob/master>, 2014, accessed in October, 2024.
- [42] "Alibaba cluster data v2020," <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2020>, 2023, accessed in October, 2024.
- [43] J. Han and L. Yan, "Adaptive batch homomorphic encryption for joint federated learning in cross-device scenarios," *IEEE IoT-J*, vol. 11, no. 6, pp. 9338–9354, 2024.
- [44] J. Liu, L. Yan, B. Li, L. Yu, and C. Shen, "SHE-loRA: Selective homomorphic encryption for federated tuning with heterogeneous loRA," in *Proc. of ICLR*, 2026.
- [45] G. Yue, L. Yan, L. Kang, and C. Shen, "Adapldp-fl: An adaptive local differential privacy for federated learning," *IEEE TMC*, vol. 24, no. 6, pp. 5569–5583, 2025.
- [46] Y. Hao, Y. Chen, J. Zakaria, B. Hu, T. Rakthanmanon, and E. Keogh, "Towards never-ending learning from time series streams," in *Proc. of SIGKDD*, 2013.
- [47] T. Rakthanmanon, B. Campana, A. Mueen, G. Batista, B. Westover, Q. Zhu, J. Zakaria, and E. Keogh, "Searching and mining trillions of time series subsequences under dynamic time warping," in *Proc. of SIGKDD*, 2012.
- [48] K. Kalpakis, D. Gada, and V. Puttagunta, "Distance measures for effective clustering of arima time-series," in *Proc. of ICDM*, 2001.
- [49] G. E. Batista, X. Wang, and E. J. Keogh, "A complexity-invariant distance measure for time series," in *Proc. of ICDM*, 2011.
- [50] J.-J. Zhu and J. Bento, "Generative adversarial active learning," *arXiv preprint arXiv:1702.07956*, 2017.
- [51] C. Nielsen and M. Okoniewski, "Gan data augmentation through active learning inspired sample acquisition," in *Proc. of CVPR Workshop*, 2019.
- [52] C. Mayer and R. Timofte, "Adversarial sampling for active learning," in *Proc. of WACV*, 2020.
- [53] Y. Liu, Z. Li, C. Zhou, Y. Jiang, J. Sun, M. Wang, and X. He, "Generative adversarial active learning for unsupervised outlier detection," *IEEE TKDE*, vol. 32, no. 8, 2020.
- [54] J. Yoon, D. Jarrett, and M. van der Schaar, "Time-series generative adversarial networks," in *Proc. of NeurIPS*, 2019.
- [55] L. Van der Maaten and G. Hinton, "Visualizing data using t-sne," *JMLR*, vol. 9, no. 11, 2008.
- [56] Q. Wen, Z. Zhang, Y. Li, and L. Sun, "Fast robuststl: Efficient and robust seasonal-trend decomposition for time series with complex patterns," in *Proc. of KDD*, 2020.
- [57] M. Yi, L. Hou, L. Shang, X. Jiang, Q. Liu, and Z.-M. Ma, "Reweight augmented samples by minimizing the maximal expected loss," *arXiv preprint arXiv:2103.08933*, 2021.
- [58] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. of ICLR*, 2015.
- [59] C.-L. Li, W.-C. Chang, Y. Cheng, Y. Yang, and B. Póczos, "Mmd gan: Towards deeper understanding of moment matching network," in *Proc. of NeurIPS*, 2017.
- [60] Q. Xu, G. Huang, Y. Yuan, C. Guo, Y. Sun, F. Wu, and K. Weinberger, "An empirical study on evaluation metrics of generative adversarial networks," *arXiv preprint arXiv:1806.07755*, 2018.

- [61] A. Borji, "Pros and cons of gan evaluation measures," *Computer Vision and Image Understanding*, vol. 179, pp. 41–65, 2019.



Xin Yong (Student Member, IEEE) received the B.S. degree in information management and information system from Shanghai University of Finance and Economics, Shanghai, China, in 2020. She is currently a Ph.D. student at the School of Cyberspace Security at Xi'an Jiaotong University. Her research interests include Federated Learning and Reinforcement Learning.



Li Yan (Member, IEEE) received the B.S. degree in information engineering from Xi'an Jiaotong University, Xi'an, China, in 2010, and the Ph.D. degree in computer science from the University of Virginia, Charlottesville, USA, in 2019.

He is currently an Associate Professor with the School of Cyber Science and Engineering, Xi'an Jiaotong University, Xi'an, China. He worked as a Postdoctoral Researcher with the Senseable City Lab, Massachusetts Institute of Technology, Cambridge, USA, from 2020 to 2021. His research interests include data-driven cyber-physical systems, big data analytics, and mobile computer networks. Dr. Yan was the Best Transactions Paper Awardee of IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS and the Best-in-Session-Presentation Awardee of INFOCOM'2017.

search interests include data-driven cyber-physical systems, big data analytics, and mobile computer networks. Dr. Yan was the Best Transactions Paper Awardee of IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS and the Best-in-Session-Presentation Awardee of INFOCOM'2017.



Yu Kuang (Student Member, IEEE) received her B.S. Degree in information security from the University of Nanjing of China in 2020. She is currently a M.S student at the School of Cyberspace Security at Xi'an Jiaotong University. Her research interests include Federated Learning and Cloud Computing.



Zhuozhao Li (Member, IEEE) received the Ph.D. degree in computer science from the University of Virginia. He is an assistant professor with the Department of Computer Science and Engineering and Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology. He was a postdoctoral scholar with the University of Chicago. His research interests span the broad areas of distributed systems, cloud computing, and high-performance computing, with the emphasis on

developing working prototypes for real-world problems and designing the foundation methodologies to optimize system performance for efficient computing.



Chao Shen (Fellow, IEEE) is currently a Professor with the School of Electronic and Information Engineering, Xi'an Jiaotong University, China, where he is also the Associate Dean of the School of Cyber Security. He is also with the Ministry of Education Key Laboratory for Intelligent Networks and Network Security. He was a Research Scholar with Carnegie Mellon University, from 2011 to 2013. His research interests include network security, human-computer interaction, insider detection, and behavioral bio-

metrics.



Xingwei Wang received the Ph.D. degree in computer science from the Northeastern University, in 1998. He is currently a Professor with the School of Computer Science and Engineering, Northeastern University. He has published more than 100 journal articles, books and book chapters, and refereed conference papers. His research interests include cloud computing and future Internet. He has received several best paper awards.